

Отчёт по лабораторной работе №5

Расширенная настройка HTTP-сервера Apache

Камбунду Паулине Де Лоурдеш Бранку

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Вывод	18
4	Контрольные вопросы	20

Список иллюстраций

2.1	Генерация закрытого RSA-ключа и самоподписанного сертификата с помощью OpenSSL	7
2.2	Конфигурация виртуальных хостов Apache для HTTP и HTTPS . .	10
2.3	Разрешение HTTPS в firewalld и перезапуск веб-сервера Apache .	11
2.4	Предупреждение Firefox об использовании самоподписанного сертификата	12
2.5	Проверка открытия сайта www.paulinelourdes.net по протоколу HTTPS	12
2.6	Просмотр сведений о самоподписанном сертификате веб-сервера	13
2.7	Создание файла <code>index.php</code> с вызовом функции <code>phpinfo</code>	14
2.8	Установка PHP, настройка прав доступа и восстановление контекстов SELinux	15
2.9	Проверка работы PHP 8.3.33 с помощью страницы <code>phpinfo</code>	16
2.10	Скрипт автоматической настройки Apache, PHP и межсетевого экрана	17

Список таблиц

1 Цель работы

Приобретение практических навыков по расширенному конфигурированию HTTP-сервера Apache в части безопасности и возможности использования PHP.

2 Выполнение лабораторной работы

Для организации защищённого доступа к веб-серверу на виртуальной машине `server` выполняю настройку Apache для работы по протоколу HTTPS. Для хранения закрытого ключа используется каталог `/etc/pki/tls/private`, связанный с каталогом `/etc/ssl/private`, а сертификат размещается в каталоге сертификатов `/etc/ssl/certs`.

Перехожу в каталог закрытых ключей и с помощью OpenSSL создаю новый закрытый RSA-ключ и самоподписанный сертификат X.509. Для этого выполняю команду:

```
openssl req -x509 -nodes -newkey rsa:2048 -keyout www.paulinelourdes.net.key  
-out www.paulinelourdes.net.crt
```

Параметр `req` запускает механизм создания запроса и сертификата, параметр `-x509` указывает на необходимость формирования самоподписанного сертификата X.509, а `-nodes` отключает шифрование закрытого ключа парольной фразой. Это необходимо для того, чтобы Apache мог использовать закрытый ключ автоматически при запуске, не запрашивая пароль у администратора. Параметр `-newkey rsa:2048` одновременно создаёт новый закрытый ключ RSA длиной 2048 бит. Параметр `-keyout` определяет имя файла закрытого ключа `www.paulinelourdes.net.key`, а `-out` — имя создаваемого сертификата `www.paulinelourdes.net.crt`.

Во время создания сертификата OpenSSL запрашивает сведения о владельце сертификата. В поле кода страны указываю `RU`, в поле региона — `Russia`, в поле города — `Moscow`, а в качестве организации и подразделения использует-

ся идентификатор `pauline@lourdes`. Для сертификата также задаются доменное имя и адрес электронной почты. Большое количество символов `+`, `.` и `*` в выводе OpenSSL отображает процесс генерации криптографического материала для нового RSA-ключа (см. рис. Рисунок 2.1).

[illegible]

Рисунок 2.1: Генерация закрытого RSA-ключа и самоподписанного сертификата с помощью OpenSSL

После создания ключа и сертификата сертификат размещаю в каталоге `/etc/ssl/certs`, а закрытый ключ оставляю в `/etc/ssl/private`. Далее перехожу в каталог `/etc/httpd/conf.d` и редактирую конфигурационный файл виртуального веб-сервера `/etc/httpd/conf.d/www.paulinelourdes.net.conf`. Конфигурация содержит два виртуальных хоста: первый обрабатывает обычные HTTP-соединения на порту 80 и перенаправляет их на HTTPS, второй обслуживает защищённые HTTPS-соединения на порту 443.

В результате файл имеет следующее содержимое:

```
<VirtualHost *:80>
    ServerAdmin webmaster@paulinelourdes.net
    DocumentRoot /var/www/html/www.paulinelourdes.net
    ServerName www.paulinelourdes.net
    ServerAlias www.paulinelourdes.net
    ErrorLog logs/www.paulinelourdes.net-error_log
```

```

    CustomLog logs/www.paulinelourdes.net-access_log common
    RewriteEngine on
    RewriteRule ^(.*)$ https://%{HTTP_HOST}$1 [R=301,L]
</VirtualHost>

<IfModule mod_ssl.c>
<VirtualHost *:443>
    SSLEngine on
    ServerAdmin webmaster@paulinelourdes.net
    DocumentRoot /var/www/html/www.paulinelourdes.net
    ServerName www.paulinelourdes.net
    ServerAlias www.paulinelourdes.net
    ErrorLog logs/www.paulinelourdes.net-error_log
    CustomLog logs/www.paulinelourdes.net-access_log common
    SSLCertificateFile /etc/ssl/certs/www.paulinelourdes.net.crt
    SSLCertificateKeyFile /etc/ssl/private/www.paulinelourdes.net.key
</VirtualHost>
</IfModule>

```

Строка `<VirtualHost *:80>` открывает описание виртуального хоста, принимающего соединения на всех сетевых интерфейсах сервера по стандартному HTTP-порту 80. Директива `ServerAdmin webmaster@paulinelourdes.net` задаёт контактный адрес администратора веб-сервера. Директива `DocumentRoot /var/www/html/www.paulinelourdes.net` определяет каталог, из которого Apache выдаёт содержимое сайта. `ServerName www.paulinelourdes.net` устанавливает основное доменное имя виртуального хоста, а `ServerAlias www.paulinelourdes.net` задаёт дополнительное имя, по которому может быть выбран данный виртуальный хост.

Директива `ErrorLog logs/www.paulinelourdes.net-error_log` задаёт отдель-

ный журнал ошибок веб-сервера, а CustomLog logs/www.paulinelourdes.net-access_log common — журнал обращений к сайту в стандартном формате common. Строка RewriteEngine on включает механизм преобразования URL модуля mod_rewrite. Следующая директива RewriteRule ^(.*)\$ https://%{HTTP_HOST}\$1 [R=301,L] формирует перенаправление каждого HTTP-запроса на соответствующий HTTPS-адрес. Переменная %{HTTP_HOST} сохраняет имя запрошенного узла, \$1 переносит совпавшую часть пути, флаг R=301 задаёт постоянное HTTP-перенаправление с кодом 301, а L прекращает дальнейшую обработку правил после срабатывания текущего. Строка </VirtualHost> завершает описание виртуального хоста HTTP.

Конструкция <IfModule mod_ssl.c> указывает Apache применять содержащиеся в ней настройки только при наличии загруженного модуля mod_ssl. Строка <VirtualHost *:443> открывает конфигурацию виртуального хоста, обслуживающего HTTPS-соединения на стандартном порту 443. Директива SSLEngine on включает использование SSL/TLS для данного виртуального хоста. Параметры ServerAdmin, DocumentRoot, ServerName, ServerAlias, ErrorLog и CustomLog выполняют те же функции, что и для виртуального хоста на порту 80.

Директива SSLCertificateFile /etc/ssl/certs/www.paulinelourdes.net.crt указывает Apache путь к созданному открытому сертификату веб-сервера. Директива SSLCertificateKeyFile /etc/ssl/private/www.paulinelourdes.net.key задаёт путь к соответствующему закрытому RSA-ключу. Строка </VirtualHost> завершает описание HTTPS-виртуального хоста, а </IfModule> завершает условный блок модуля SSL. В результате HTTP-запросы перенаправляются на HTTPS, после чего Apache использует созданные сертификат и закрытый ключ для установки защищённого соединения (см. рис. Рисунок 2.2).



```

1 <VirtualHost *:80>
2     ServerAdmin webmaster@paulinelourdes.net
3     DocumentRoot /var/www/html/www.paulinelourdes.net
4     ServerName www.paulinelourdes.net
5     ServerAlias www.paulinelourdes.net
6     ErrorLog logs/www.paulinelourdes.net-error_log
7     CustomLog logs/www.paulinelourdes.net-access_log common
8     RewriteEngine on
9     RewriteRule ^(.*)$ https://%{HTTP_HOST}%1 [R=301,L]
10 </VirtualHost>
11
12 <IfModule mod_ssl.c>
13 <VirtualHost *:443>
14     SSLEngine on
15     ServerAdmin webmaster@paulinelourdes.net
16     DocumentRoot /var/www/html/www.paulinelourdes.net
17     ServerName www.paulinelourdes.net
18     ServerAlias www.paulinelourdes.net
19     ErrorLog logs/www.paulinelourdes.net-error_log
20     CustomLog logs/www.paulinelourdes.net-access_log common
21     SSLCertificateFile /etc/ssl/certs/www.paulinelourdes.net.crt
22     SSLCertificateKeyFile /etc/ssl/private/www.paulinelourdes.net.key
23 </VirtualHost>
24 </IfModule>

```

Рисунок 2.2: Конфигурация виртуальных хостов Apache для HTTP и HTTPS

После изменения конфигурации разрешаю входящие HTTPS-соединения в межсетевом экране firewalld. Команда `firewall-cmd --add-service=https` разрешает сервис https в текущей конфигурации межсетевого экрана, а `firewall-cmd --add-service=https --permanent` сохраняет разрешение в постоянной конфигурации. В обоих случаях сообщение success подтверждает успешное выполнение команд.

Затем выполняю `firewall-cmd --reload`, чтобы перезагрузить постоянную конфигурацию firewalld и применить внесённые изменения. После настройки межсетевого экрана перезапускаю Apache командой `systemctl restart httpd`. Отсутствие сообщений об ошибках после выполнения `systemctl restart httpd` указывает на успешное принятие новой конфигурации веб-сервером (см. рис. Рисунок 2.3).

```
[root@server.paulinelourdes.net private]#  
[root@server.paulinelourdes.net private]# cd /etc/httpd/conf.d/  
[root@server.paulinelourdes.net conf.d]# gedit www.paulinelourdes.net.conf  
[root@server.paulinelourdes.net conf.d]#  
[root@server.paulinelourdes.net conf.d]# firewall-cmd --add-service=https  
success  
[root@server.paulinelourdes.net conf.d]# firewall-cmd --add-service=https --permanent  
success  
[root@server.paulinelourdes.net conf.d]# firewall-cmd --reload  
success  
[root@server.paulinelourdes.net conf.d]# systemctl restart httpd  
[root@server.paulinelourdes.net conf.d]#
```

Рисунок 2.3: Разрешение HTTPS в firewalld и перезапуск веб-сервера Apache

Для проверки работы HTTPS на виртуальной машине client открываю в браузере адрес `www.paulinelourdes.net`. Настроенное правило `mod_rewrite` переводит обращение на защищённое HTTPS-соединение. При первом подключении Firefox отображает предупреждение `Warning: Potential Security Risk Ahead`. В подробностях указано, что сертификат не является доверенным, поскольку он самоподписанный: `The certificate is not trusted because it is self-signed`. Браузер также отображает код ошибки `MOZILLA_PKIX_ERROR_SELF_SIGNED_CERT`.

Появление данного предупреждения ожидаемо, поскольку сертификат был создан непосредственно на сервере и не подписан внешним доверенным центром сертификации. Для продолжения работы открываю дополнительные параметры и выбираю `Accept the Risk and Continue`, добавляя исключение для используемого сертификата (см. рис. Рисунок 2.4).

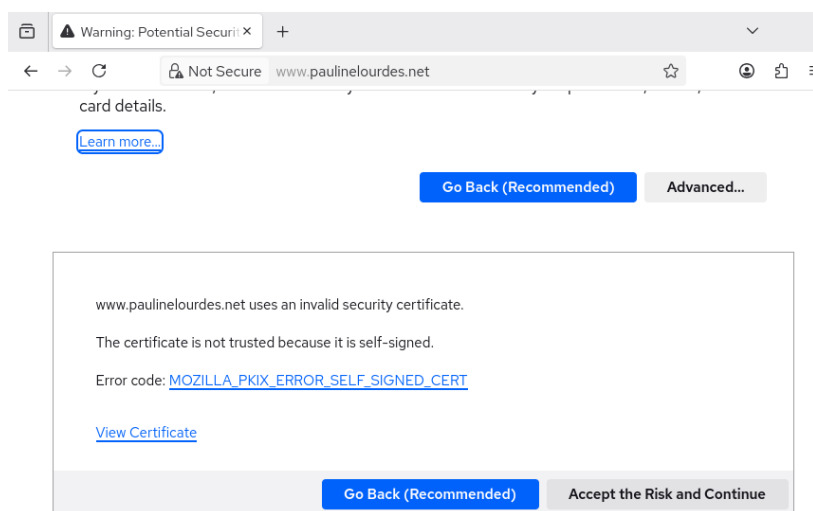


Рисунок 2.4: Предупреждение Firefox об использовании самоподписанного сертификата

После добавления исключения страница виртуального веб-сервера успешно открывается. В адресной строке отображается `www.paulinelourdes.net`, а на странице выводится ранее созданное сообщение `Welcome to the www.paulinelourdes.net server`. Наличие значка защищённого соединения возле адреса показывает, что обращение к веб-серверу выполняется с использованием HTTPS. Следовательно, перенаправление с HTTP на HTTPS и обслуживание сайта на порту 443 функционируют корректно (см. рис. Рисунок 2.5).



Рисунок 2.5: Проверка открытия сайта `www.paulinelourdes.net` по протоколу HTTPS

Дополнительно просматриваю сведения о сертификате в Firefox. В разделе `Subject Name` отображаются введённые при его создании данные: страна

RU, регион Russia, город Moscow, организация paulinelourdes, подразделение paulinelourdes, общее имя paulinelourdes.net и адрес электронной почты paulinelourdes@paulinelourdes.net. Эти сведения подтверждают, что веб-сервер использует созданный на предыдущем этапе сертификат (см. рис. Рисунок 2.6).

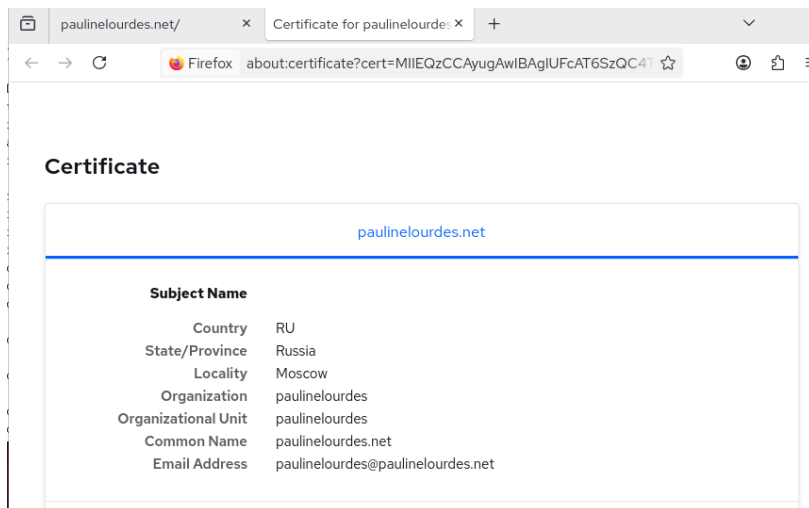


Рисунок 2.6: Просмотр сведений о самоподписанном сертификате веб-сервера

На следующем этапе настраиваю HTTP-сервер для обработки PHP. Устанавливаю PHP и необходимые зависимости командой `dnf -y install php`. В результате устанавливается PHP версии 8.3.33, а также связанные с ним пакеты `php-cli`, `php-common`, `php-fpm`, `php-mbstring`, `php-opcache`, `php-pdo` и `php-xml`. Дополнительно устанавливаются требуемые зависимости, в том числе `capstone` и `nginxfilesystem`. Сообщение `Complete!` подтверждает успешное завершение установки.

Для проверки обработки PHP заменяю существующую стартовую HTML-страницу виртуального сервера PHP-страницей. Файл `/var/www/html/www.paulinelourdes.net` переименовываю в `/var/www/html/www.paulinelourdes.net/index.php`, после чего открываю его на редактирование. Содержимое файла заменяю простым PHP-скриптом:

```
<?php
    phpinfo();
?>
```

Функция `phpinfo()` формирует диагностическую страницу с информацией об установленной версии PHP, способе взаимодействия с веб-сервером, используемых конфигурационных файлах, подключённых модулях и параметрах среды. Этот скрипт позволяет проверить, что Apache передаёт PHP-файл соответствующему обработчику, а не выдаёт исходный код клиенту (см. рис. Рисунок 2.7).

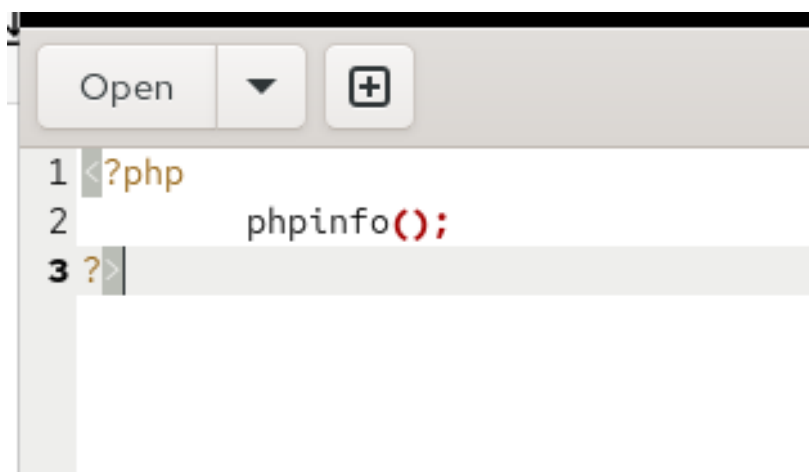


Рисунок 2.7: Создание файла `index.php` с вызовом функции `phpinfo`

После изменения стартовой страницы корректирую владельца файлов веб-контента командой `chown -R apache:apache /var/www/`. В результате владельцем и группой каталога `/var/www` и расположенных внутри него объектов становится системная учётная запись `apache`, от имени которой веб-сервер получает доступ к содержимому сайта.

Далее восстанавливаю корректные контексты безопасности SELinux командами `restorecon -vR /etc` и `restorecon -vR /var/www/`. Параметр `-R` обеспечивает рекурсивную обработку каталогов, а `-v` выводит информацию

об изменениях. В процессе обработки `/etc` система, в частности, восстанавливает контекст файла сетевого соединения `/etc/NetworkManager/system-connections/eth1.nmconnection`. В завершение выполняю `systemctl restart httpd`, чтобы Apache начал работать с установленным PHP и обновлённым содержимым веб-каталога (см. рис. Рисунок 2.8).

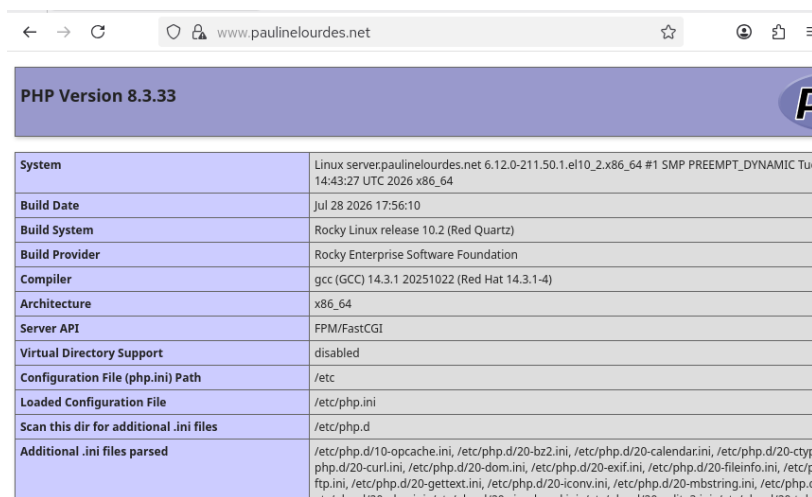
```
Installed:
capstone-5.0.1-8.el10_2.x86_64      nginxfilesystem-2:1.26.3-6.el10_2.7.noarch  php-8.3.33-1.el10_2.x86_64
php-cli-8.3.33-1.el10_2.x86_64      php-common-8.3.33-1.el10_2.x86_64          php-fpm-8.3.33-1.el10_2.x86_64
php-mbstring-8.3.33-1.el10_2.x86_64  php-opcache-8.3.33-1.el10_2.x86_64         php-pdo-8.3.33-1.el10_2.x86_64
php-xml-8.3.33-1.el10_2.x86_64

Complete!
[root@server.paulinelourdes.net conf.d]# mv /var/www/html/www.paulinelourdes.net/index.html /var/www/html/www.paulinelourdes.net/index.php
[root@server.paulinelourdes.net conf.d]# gedit /var/www/html/www.paulinelourdes.net/index.php
[root@server.paulinelourdes.net conf.d]# chown -R apache:apache /var/www/
[root@server.paulinelourdes.net conf.d]# restorecon -vR /etc
Relabeled /etc/NetworkManager/system-connections/eth1.nmconnection from unconfined_u:object_r:user_tmp_t:s0 to unconfined_u:object_r:NetworkManager_etc_rw_t:s0
[root@server.paulinelourdes.net conf.d]# restorecon -vR /var/www/
[root@server.paulinelourdes.net conf.d]# systemctl restart httpd
[root@server.paulinelourdes.net conf.d]#
```

Рисунок 2.8: Установка PHP, настройка прав доступа и восстановление контекстов SELinux

Для проверки результата на виртуальной машине `client` снова открываю `www.paulinelourdes.net`. Вместо исходной статической страницы браузер отображает результат выполнения функции `phpinfo()`. В верхней части страницы указана версия PHP 8.3.33. В таблице также видно, что сервер работает под Rocky Linux 10.2, используется архитектура `x86_64`, а в поле `Server API` указано `FPM/FastCGI`. Основной конфигурационный файл PHP загружен из `/etc/php.ini`, а дополнительные файлы конфигурации считываются из каталога `/etc/php.d`.

Появление сформированной PHP страницей информации подтверждает, что PHP установлен, интегрирован с веб-сервером и корректно выполняет файлы с расширением `.php` (см. рис. Рисунок 2.9).



PHP Version 8.3.33	
System	Linux server.paulinelourdes.net 6.12.0-211.50.1.el10_2.x86_64 #1 SMP PREEMPT_DYNAMIC Tue 14:43:27 UTC 2026 x86_64
Build Date	Jul 28 2026 17:56:10
Build System	Rocky Linux release 10.2 (Red Quartz)
Build Provider	Rocky Enterprise Software Foundation
Compiler	gcc (GCC) 14.3.1 20251022 (Red Hat 14.3.1-4)
Architecture	x86_64
Server API	FPM/FastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc
Loaded Configuration File	/etc/php.ini
Scan this dir for additional .ini files	/etc/php.d
Additional .ini files parsed	/etc/php.d/10-opcache.ini, /etc/php.d/20-bz2.ini, /etc/php.d/20-calendar.ini, /etc/php.d/20-ctype, /etc/php.d/20-curl.ini, /etc/php.d/20-dom.ini, /etc/php.d/20-exif.ini, /etc/php.d/20-fileinfo.ini, /etc/pl, /etc/php.d/20-gettext.ini, /etc/php.d/20-iconv.ini, /etc/php.d/20-mbstring.ini, /etc/php.d, /etc/php.d/20-phar.ini, /etc/php.d/20-simplexml.ini, /etc/php.d/20-sockets.ini, /etc/php.d/20-tokenizer, /etc/php.d/20-xml.ini, /etc/php.d/20-xmlrpc.ini

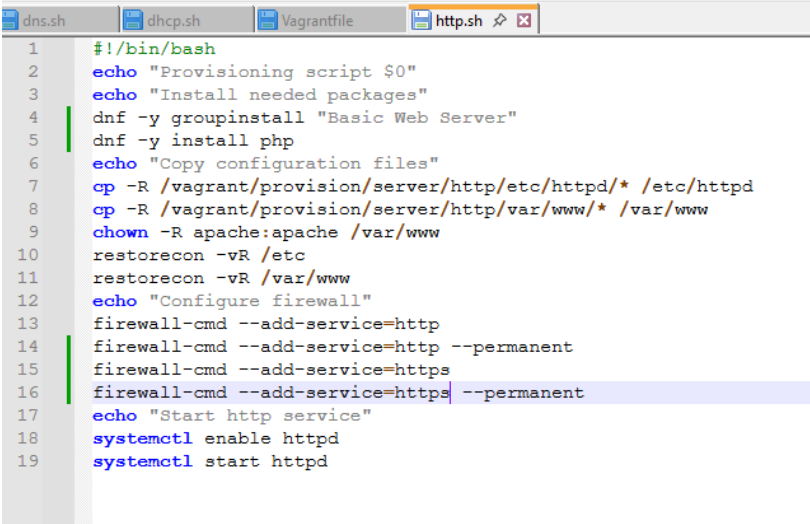
Рисунок 2.9: Проверка работы PHP 8.3.33 с помощью страницы phpinfo

Для фиксации выполненной расширенной настройки виртуальной машины server в сценарии автоматического развёртывания вношу изменения в `/vagrant/provision/server/http.sh`. Скрипт содержит установку группы пакетов базового веб-сервера и дополнительно устанавливает PHP командой `dnf -y install php`. После этого из каталога `/vagrant/provision/server/http` копируются сохранённые конфигурационные файлы Apache и содержимое `/var/www`, для веб-каталога устанавливаются владелец и группа `apache:apache`, а командами `restorecon -vR /etc` и `restorecon -vR /var/www` восстанавливаются контексты SELinux.

В секции настройки межсетевого экрана сохранены команды разрешения обычного HTTP-трафика `firewall-cmd --add-service=http` и `firewall-cmd --add-service=http --permanent`. Дополнительно в сценарий внесены команды `firewall-cmd --add-service=https` и `firewall-cmd --add-service=https --permanent`, разрешающие HTTPS-соединения как в текущей, так и в постоянной конфигурации `firewalld`.

В конце сценария выполняются `systemctl enable httpd` и `systemctl start httpd`. Первая команда включает автоматический запуск службы Apache при загрузке виртуальной машины, а вторая непосредственно запускает

веб-сервер. Благодаря внесённым изменениям сценарий автоматизации устанавливает как Apache, так и PHP, разворачивает сохранённую конфигурацию веб-сервера, восстанавливает параметры доступа и SELinux и разрешает работу по протоколам HTTP и HTTPS (см. рис. Рисунок 2.10).



```
1  #!/bin/bash
2  echo "Provisioning script $0"
3  echo "Install needed packages"
4  dnf -y groupinstall "Basic Web Server"
5  dnf -y install php
6  echo "Copy configuration files"
7  cp -R /vagrant/provision/server/http/etc/httpd/* /etc/httpd
8  cp -R /vagrant/provision/server/http/var/www/* /var/www
9  chown -R apache:apache /var/www
10 restorecon -vR /etc
11 restorecon -vR /var/www
12 echo "Configure firewall"
13 firewall-cmd --add-service=http
14 firewall-cmd --add-service=http --permanent
15 firewall-cmd --add-service=https
16 firewall-cmd --add-service=https --permanent
17 echo "Start http service"
18 systemctl enable httpd
19 systemctl start httpd
```

Рисунок 2.10: Скрипт автоматической настройки Apache, PHP и межсетевого экрана

3 Вывод

В ходе лабораторной работы была выполнена расширенная настройка HTTP-сервера Apache на виртуальной машине server. Основное внимание уделялось переходу веб-сервера от использования обычного протокола HTTP к защищённому протоколу HTTPS, подключению поддержки PHP и внесению выполненных настроек в сценарий автоматического развёртывания виртуальной машины.

Для организации защищённого соединения с помощью утилиты `openssl` были сгенерированы закрытый RSA-ключ длиной 2048 бит и самоподписанный сертификат X.509 для веб-сервера `www.paulinelourdes.net`. Закрытый ключ был размещён в каталоге `/etc/ssl/private`, а сертификат — в каталоге `/etc/ssl/certs`. Использование параметра `-nodes` позволило создать закрытый ключ без парольной фразы, благодаря чему Apache может автоматически использовать его при запуске без дополнительного ввода пароля.

Конфигурационный файл виртуального хоста `/etc/httpd/conf.d/www.paulinelourdes.net.conf` был изменён для поддержки HTTPS. Для виртуального хоста на порту 80 было настроено автоматическое перенаправление HTTP-запросов на HTTPS с использованием модуля `mod_rewrite`. Для виртуального хоста на порту 443 был включён механизм SSL/TLS директивой `SSLEngine on`, а также указаны пути к созданному сертификату и закрытому ключу с помощью директив `SSLCertificateFile` и `SSLCertificateKeyFile`.

Для обеспечения сетевого доступа по защищённому протоколу в межсетевом экране `firewalld` был разрешён сервис `https`. После внесения изменений

конфигурация межсетевого экрана была перезагружена, а служба `httpd` перезапущена. Проверка с виртуальной машины `client` показала, что при обращении к `www.paulinelourdes.net` происходит переход на HTTPS. Поскольку использовался самоподписанный сертификат, браузер первоначально вывел предупреждение о недоверенном сертификате. После добавления исключения сайт успешно открылся по защищённому соединению, а просмотр сведений о сертификате подтвердил использование созданного сертификата веб-сервера.

На следующем этапе был установлен PHP и необходимые для его работы компоненты. Статическая стартовая страница веб-сервера была заменена файлом `index.php`, содержащим вызов функции `phpinfo()`. Для каталога `/var/www` был установлен владелец `apache:apache`, после чего восстановлены контексты безопасности SELinux. После перезапуска Apache на клиентской машине успешно отобразилась диагностическая страница PHP. На ней была указана установленная версия PHP 8.3.33, что подтвердило корректную обработку PHP-сценариев веб-сервером.

Для автоматизации расширенной настройки сервера были внесены изменения в provisioning-сценарий `/vagrant/provision/server/http.sh`. В него была добавлена установка PHP, копирование конфигурационных файлов Apache, веб-контента, сертификата и закрытого ключа, восстановление контекстов SELinux, настройка прав доступа, а также разрешение сервисов `http` и `https` в `firewalld`. Скрипт также включает службу Apache в автозагрузку и запускает её. В результате созданный сценарий позволяет автоматически восстановить настроенную конфигурацию веб-сервера при повторном развёртывании виртуальной машины.

4 Контрольные вопросы

1. В чём отличие HTTP от HTTPS?

HTTP (HyperText Transfer Protocol) — прикладной протокол, предназначенный для передачи веб-страниц и других данных между клиентом и веб-сервером. При обычном HTTP передаваемые данные сами по себе не защищены средствами протокола: информация передаётся без шифрования и при наличии доступа к сетевому трафику потенциально может быть прочитана или изменена третьей стороной.

По умолчанию HTTP использует TCP-порт:

80

HTTPS (HyperText Transfer Protocol Secure) представляет собой использование HTTP поверх защищённого соединения TLS (Transport Layer Security). Перед началом передачи HTTP-данных между клиентом и сервером устанавливается защищённый TLS-сеанс, в рамках которого выполняется проверка сертификата сервера и согласование криптографических параметров соединения.

По умолчанию HTTPS использует TCP-порт:

443

Основное отличие состоит в том, что HTTPS обеспечивает шифрование передаваемых данных, проверку их целостности и возможность подтверждения подлинности веб-сервера с помощью цифрового сертификата.

Упрощённо различие можно представить следующим образом:

HTTP = HTTP без встроенной криптографической защиты

HTTPS = HTTP + TLS

В выполненной лабораторной работе обращения к веб-серверу на порту 80 автоматически перенаправлялись на HTTPS, после чего дальнейший обмен данными выполнялся через защищённое соединение на порту 443.

2. Каким образом обеспечивается безопасность контента веб-сервера при работе через HTTPS?

Безопасность передачи данных при использовании HTTPS обеспечивается протоколом **TLS**. Он создаёт защищённый канал связи между браузером клиента и веб-сервером.

TLS обеспечивает три основных свойства защищённого соединения:

- **конфиденциальность** — передаваемые данные шифруются, поэтому перехваченный сетевой трафик нельзя непосредственно прочитать без соответствующих криптографических ключей;
- **целостность** — TLS позволяет обнаружить изменение данных во время их передачи между клиентом и сервером;
- **аутентификацию сервера** — клиент получает цифровой сертификат веб-сервера и может проверить, действительно ли он устанавливает соединение с требуемым узлом.

При установлении HTTPS-соединения сервер предоставляет клиенту свой цифровой сертификат. Сертификат содержит сведения о владельце, доменном имени, открытом ключе, сроке действия и других параметрах. Закрытый ключ хранится только на сервере и не передаётся клиентам.

В лабораторной работе для веб-сервера были созданы:

`www.paulinelourdes.net.crt` — цифровой сертификат;

`www.paulinelourdes.net.key` — закрытый криптографический ключ.

В конфигурации Apache они подключены директивами:

```
SSLCertificateFile /etc/ssl/certs/www.paulinelourdes.net.crt
```

```
SSLCertificateKeyFile /etc/ssl/private/www.paulinelourdes.net.key
```

При начале соединения клиент и сервер выполняют TLS-рукопожатие. В процессе него сервер передаёт сертификат клиенту, стороны согласовывают поддерживаемые криптографические алгоритмы и формируют общий сеансовый ключ. Дальнейший обмен данными выполняется с использованием симметричного шифрования, поскольку оно значительно эффективнее для передачи больших объёмов информации.

Сертификат, использованный в лабораторной работе, является **самоподписанным**. Он также позволяет организовать зашифрованный канал передачи данных, однако его подлинность не подтверждена доверенным сертификационным центром. Поэтому браузер при первом подключении отображает предупреждение о недоверенном сертификате.

В реальных публичных веб-сервисах обычно используются сертификаты, подписанные доверенным сертификационным центром. В этом случае браузер может автоматически проверить цепочку доверия и подтвердить принадлежность сертификата соответствующему серверу.

Следует учитывать, что HTTPS защищает прежде всего данные **во время передачи между клиентом и сервером**. Он не обеспечивает автоматическую защиту файлов веб-сервера от неправильных прав доступа, уязвимостей веб-приложения, вредоносных программ или получения злоумышленником доступа к самому серверу. Поэтому HTTPS используется совместно с другими механизмами защиты операционной системы и веб-приложений.

3. Что такое сертификационный центр? Приведите пример.

Сертификационный центр, или **центр сертификации** (Certificate Authority, CA), — организация или специализированная служба, которая выпускает и подписывает цифровые сертификаты.

Основная задача сертификационного центра заключается в подтверждении связи между открытым ключом, содержащимся в сертификате, и владельцем сертификата, например определённым доменным именем.

Когда владелец веб-сервера получает сертификат от доверенного центра сертификации, СА подписывает сертификат собственной цифровой подписью. Браузер может проверить эту подпись с помощью цепочки доверия. Корневые сертификаты известных центров сертификации заранее включены в хранилища доверенных сертификатов операционных систем и браузеров.

Упрощённо цепочка проверки может выглядеть следующим образом:

Корневой сертификат СА → промежуточный сертификат СА → сертификат веб-сервера

Если цепочка корректна, сертификат не просрочен, доменное имя соответствует указанному в сертификате и сертификат не был отозван, браузер может считать соединение доверенным.

Примером центра сертификации является **Let's Encrypt**. Он предоставляет бесплатные TLS-сертификаты и позволяет автоматизировать их выпуск и продление. Сертификаты Let's Encrypt широко используются для организации HTTPS на публичных веб-серверах.

Также к известным центрам сертификации относятся:

DigiCert

GlobalSign

Sectigo

В данной лабораторной работе внешний сертификационный центр не использовался. Сертификат был создан и подписан собственным закрытым ключом, поэтому являлся самоподписанным. Из-за отсутствия доверенной цепочки сертификации браузер Firefox вывел предупреждение MOZILLA_PKIX_ERROR_SELF_SIGNED_CERT, после чего сертификат был принят вручную в качестве исключения.